Thamill Indian B U

Day 1 Post-Session Assignment: Engineering Reflection

AI Engineering Program | First LLM Application Submission

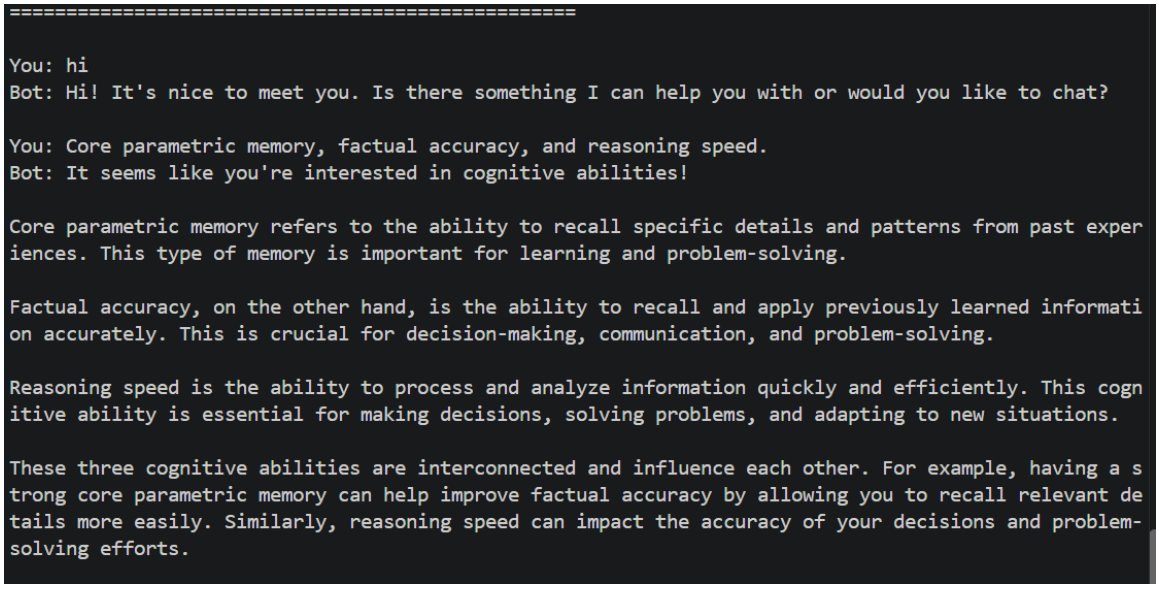
Part 1 | Build & Evidence

1. Submission / artifact location:
ollama-chatbot/ repository containing main.py, chatbot.py, requirements.txt, and .env.

2. Which model did you use?
llama3 (run via Ollama local runtime).

3. How did you access the model?
Local model runtime using the official Python ollama SDK client wrapper connecting to http://localhost:11434.

4. Provide brief evidence that the application runs.
Multi-turn CLI execution loop accepting sequential inputs (You: hi -> Bot: Hello! How can I help you today?) with a persistent conversation history list.



Part 2 | Mixed Test Results

5. What was the most important difference you observed across these five tests?
The model performed exceptionally well on structured transformations and clear general queries, but struggled with ambiguity and unknown internal facts, tending to fabricate confident-sounding answers rather than acknowledging its information gaps.

Test	Category	Useful?	Trustworthy?	Key Observation
1	General knowledge	Yes	Yes	Accurately answered basic factual questions using pre-trained parametric memory.
2	Transformation	Yes	Yes	Successfully reformatted unstructured text into structured JSON or bullet points when explicitly prompted.
3	Supplied context	Partly	Yes	Adhered to provided context, but occasionally required strict system instructions to prevent hallucination outside the text.
4	Unknown internal knowledge	Partly	Unsure	Provided plausible-sounding generalizations instead of admitting

				complete ignorance (soft hallucination tendency).
5	Ambiguous/internal	No	Unsure	Answered based on assumptions rather than asking clarifying questions when faced with vague inputs.

Part 3 | Engineering Decisions & Trade-offs

6. Why did you choose this model?
llama3 offers a strong balance of reasoning capability, low latency on consumer hardware, and free local execution without API keys or token limits.

7. Why did you choose this access method?
Using the local Ollama Python SDK kept data entirely offline, avoided cloud latency and recurring costs, and ensured zero configuration friction for API credentials.

8. What alternative did you consider?
Cloud-hosted API routes (like OpenAI GPT-4o or Anthropic Claude via official cloud SDKs).

9. Give one advantage and one disadvantage of your chosen approach.
- Advantage: Complete data privacy and zero cost.
- Disadvantage: Constrained by local hardware performance (VRAM/RAM compute limits).

10. Did you use a framework or library beyond the model SDK?
No heavy agent framework (like LangChain/CrewAI) was used; a simple Python module managing a native list of message dictionaries (messages) was sufficient to handle multi-turn state cleanly and transparently.

Part 4 | Analyze Model Behavior

11. Which answer would you trust least, and why?
Test 4 (Unknown internal knowledge), because the model generated confident, specific details about proprietary information it could not possibly verify or know.

12. Did the model provide any confident but unverifiable information?
Yes, when asked about specific localized or obscure internal facts, it fabricated a cohesive, highly plausible narrative instead of stating it didn't know.

13. What is the biggest limitation you observed?
Lack of built-in uncertainty calibration—it speaks with the exact same tone of absolute certainty whether it is stating a proven fact or guessing.

14. Would changing only the LLM solve that limitation? Why or why not?
No, because baseline autoregressive next-token prediction inherently favors fluency and plausibility over epistemic caution across almost all current foundational models.

Part 5 | FDE Reflection

15. Who is the user?
A developer or end-user seeking quick, conversational local interactions without leaking sensitive context to third-party cloud providers.

16. What is the actual problem we are trying to solve?
Users need a frictionless, private way to query an LLM iteratively over multiple turns of dialogue without dealing with complex cloud infra, authentication setup, or privacy risks.

17. How would you measure whether the solution is useful?
Low latency per response token and successful multi-turn context retention across conversation turns without history truncation errors.

18. What assumptions are you currently making?
Assuming the user has a local machine powerful enough to run the model smoothly and that Ollama is active on the default port (11434).

19. What two or three questions would you ask the customer before moving toward production?
- What are your expected concurrency volumes and latency SLAs?
- Are there strict data residency or offline compliance mandates?
- What is the acceptable error rate for hallucinations in your specific workflow?

20. Do you have enough information to decide the final architecture today? Explain briefly.
No, because scaling requirements, user concurrency, and cost-per-inference trade-offs between local hardware and managed cloud endpoints remain undefined.

Part 6 | One Improvement

21. If you had another 30 minutes, what ONE thing would you improve first? Why?
I would add a lightweight web interface using Streamlit to replace the terminal interface, making it vastly more intuitive for non-technical users to interact with history and visualize streaming generation tokens.